

Model-View表單的資料繫結

- 8-1: Form的資料繫結

Request.Form	8-2
方法參數接收	8-4
FromForm	8-5

- 8-2: Model Binding模型繫結

複雜模型繫結	8-6
HttpGet與HttpPost差異	8-8
以強型別呈現資料	8-9

- 8-3: Data Annotations

顯示用Data Annotations	8-10
驗證用Data Annotations	8-11
驗證用Tag Helpers	8-13

Module 08

Request.Form -1

- Request.Form使用方式與字典相似，每個Key均對應到一個Value。
- Request.Form的Key會與Form表單中HTML標籤的name屬性繫結在一起。



Request.Form -2

- 使用Request.Form將表單中的資料繫結。

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="PostLogin" method="post">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email" class="control-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Password" class="control-label"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
      </div>
      <div class="form-group">
        <input type="submit" value="登入" class="btn btn-primary" />
      </div>
    </form>
  </div>
</div>
```

```
[HttpPost]
0 個參考
public IActionResult PostLogin()
{
    ViewBag.Message = Request.Form["email"];
    return View("Index");
}
```

方法參數接收

- 使用方法參數接收Form表單資料。

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="PostLogin" method="post">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email" class="control-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Password" class="control-label"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
      </div>
      <div class="form-group">
        <input type="submit" value="登入" class="btn btn-primary" />
      </div>
    </form>
  </div>
</div>
```

```
[HttpPost]
0 個參考
public IActionResult PostLogin(string Email , string Password)
{
    ViewBag.Message = Email;
    return View("Index");
}
```

FromForm

- 於參數前加入定義[FromForm]，常用於 API Controller。

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="PostLogin" method="post">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email" class="control-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Password" class="control-label"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
      </div>
      <div class="form-group">
        <input type="submit" value="登入" class="btn btn-primary" />
      </div>
    </form>
  </div>
</div>
```

```
public class LoginViewModel
{
    0 個參考
    public string Email { get; set; }
    0 個參考
    public string Password { get; set; }
}
```

```
[HttpPost]
0 個參考
public IActionResult PostLogin([FromForm] string email)
{
    ViewBag.Message = email;
    return View("Index");
}
```

複雜模型繫結 -1

- 將回傳的資料欄位與類別 (Model) 中的屬性繫結。
- 建立符合回傳資料的類別(Model)，Form表單中HTML標籤的name屬性必須與類別中屬性名稱相同。
- 用強型別的方式取得資料，減少因錯字而產生的錯誤。
- 需注意HTTP Method(FormMethod)及傳送的Action是否正確。

複雜模型繫結 -2

- 新建一個類別當作參數接收Form表單資料。

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="PostLogin" method="post">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Email" class="control-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Password" class="control-label"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
      </div>
      <div class="form-group">
        <input type="submit" value="登入" class="btn btn-primary" />
      </div>
    </form>
  </div>
</div>
```

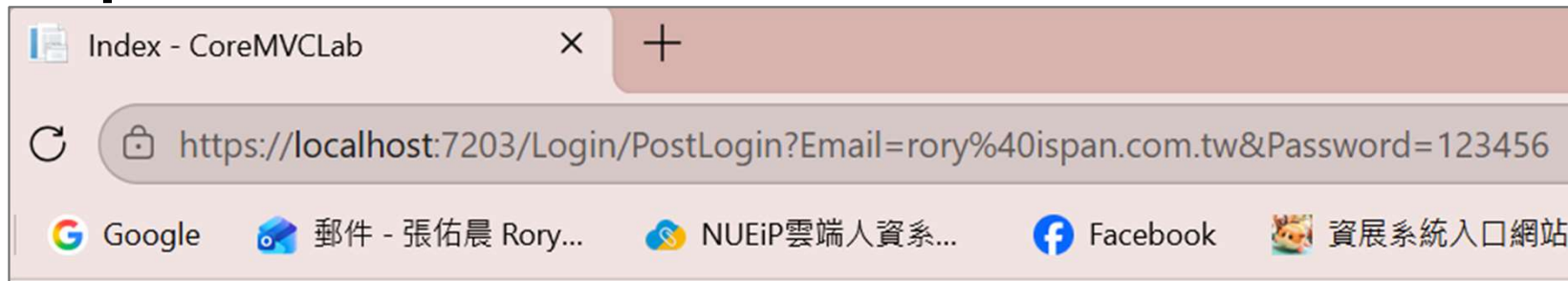
```
public class LoginViewModel
{
    0 個參考
    public string Email { get; set; }
    0 個參考
    public string Password { get; set; }
}
```

```
[HttpPost]
0 個參考
public IActionResult PostLogin(LoginViewModel login)
{
    ViewBag.Message = login.Email;
    return View("Index");
}
```

Http Get與HttpPost差異

- 表單輸入內容後比較差異。

- **Http Get :**



- **Http Post :**



以強型別呈現資料

- 利用強型別的方式顯示資料。
- 與先前談到的複雜模型繫結概念相似，都是以模型類別的方式傳送資料。

```
public IActionResult Detail(int id)
{
    return View(new Member
    {
        Id = id,
        Name = "John Doe",
        BirthDate = new DateTime(1995, 4, 1),
        Email = $"{id}@abc.com.tw"
    });
}
```

```
@model CoreMVCLab.Models.Member
<div>
    <h4>Member</h4>
    <hr />
    <div class="form-group">
        <label class="form-control" >@Model.Id</label>
        <label asp-for="Name" class="control-label"> </label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"> </span>
        <label asp-for="BirthDate" class="control-label"> </label>
        <input type="date" asp-for="BirthDate" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"> </span>
        <label asp-for="Email" class="control-label"> </label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"> </span>
    </div>
</div>
```

顯示用Data Annotations

- **[DisplayName(“自訂名稱”)]**：自訂模型屬性顯示的文字，需搭配 **Html.DisplayNameFor()** 使用。

```
[DisplayName("會員名稱")]  
13 個參考  
public string? Name { get; set; }  
9 個參考
```



```
<label asp-for="Name" class="control-label"> </label>
```

會員姓名

Email



- **[DataType(DataType. html標籤型別)]**：設定HTML標籤型別。
- **[DisplayFormat(DataFormatString=“Pattern”)]**：設定資料顯示的格式。

```
[DataType(DataType.DateTime)]  
[DisplayFormat(DataFormatString = "{0:dd/MM/yy}")]  
0 個參考  
public DateTime CreatedDate { get; set; }
```

驗證用 Data Annotations -1

- Data Annotations 可套用在 ViewModel / Model 屬性上，用於表單輸入驗證。
 - [Required]：必填欄位
 - [StringLength(max)]：限制字串長度（最大長度）
 - [Range(min, max)]：限制數值輸入範圍
 - [RegularExpression("Pattern")]：使用正規表達式驗證格式
- 可在驗證屬性中加入 ErrorMessage，當驗證失敗時顯示指定訊息。

驗證用 Data Annotations -2

```
[DisplayName("會員姓名")]
[Required(ErrorMessage = "姓名為必填欄位！")]
public string MemberName { get; set; }

[Required(ErrorMessage = "Email為必填欄位！")]
[DataType(DataType.EmailAddress)]
[EmailAddress(ErrorMessage = "Email格式錯誤！")]
public string Email { get; set; }

[DataType(DataType.Password)]
[StringLength(20)]
public string Password { get; set; }

[Range(1,100)]
public int CityID { get; set; }
```

驗證用Tag Helpers

- 驗證用Data Annotations須搭配Tag Helpers，才能將驗證失敗的錯誤訊息顯示在頁面上。
- **asp-validation-summary**：顯示所有驗證後的錯誤訊息。
 - All：顯示 所有 ModelState 錯誤（含欄位錯誤與模型層級錯誤）
 - ModelOnly：只顯示「模型層級」錯誤（不含欄位錯誤）
 - None：不顯示（等同關閉）

```
<div asp-validation-summary="All" class="text-danger"></div>
```

- **asp-validation-for**：顯示個別屬性的錯誤訊息。

```
<span asp-validation-for="Password" class="text-danger"></span>
```